



Artificial Intelligence Fundamentals

CONSTRAINT SATISFACTION PROBLEM
(CSP)

Factored state representation

So far, **atomic states**. Now, **factored representation**.

State $X = \{X_1, X_2, \dots, X_n\}$ of n variables, with $X_i \in D_i$ (the domain)

Domain knowledge is expressed by a set of **constraints C**

- e.g., $X_1 \leq 0$, $X_2 \in [1,3]$, ...

CSP(X, D, C): assigns values to variables to satisfy all constraints (goal test)

- **Complete vs. partial assignment:** all / only a few variables have been assigned a value
- **Consistent assignment:** assigned variables satisfy constraints
- **Partial solution:** a partial assignment that is consistent

CSP's objective: find a complete and consistent assignment

From search to knowledge

X, D, C can be expressed in any formal language

We can then apply search to find solutions to statements expressed in that language

Search is now a general-purpose tool to solve a wide variety of problems that can be expressed via CSP

This is becoming quite powerful

- CSP is NP-complete in general

Map coloring – 3 colors

Color adjacent regions with different colors

$X =$

- WA, NT, SA, Q, NSW, V, T

$D =$

- $\{r, g, b\} \forall X_i$

$C =$

- $WA \neq NT$ (repeat for all pairs)
- $(WA, NT) \in \{(r, g), (r, b), (g, r), (g, b), \dots\}$ (repeat for all pairs)

Complete assignment:

all states are assigned a color

Consistent assignment:

no neighbor states in the assignment share the same color



Map coloring

Color adjacent regions with different colors

$X =$

- WA, NT, SA, Q, NSW, V, T

$D =$

- $\{r, g, b\} \ \forall X_i$

$C =$

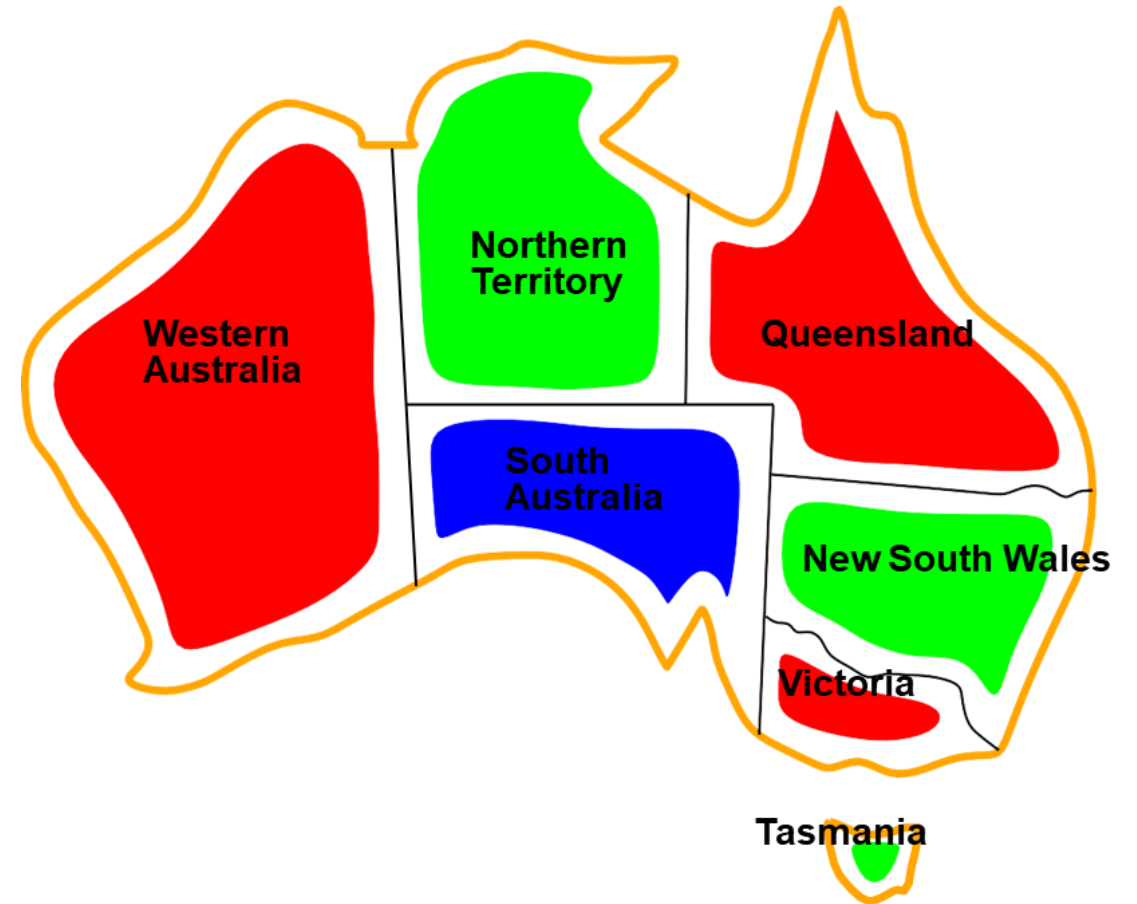
- $WA \neq NT$ (repeat for all pairs)
- $(WA, NT) \in \{(r, g), (r, b), (g, r), (g, b), \dots\}$ (repeat for all pairs)

Complete assignment:

all states are assigned a color

Consistent assignment:

no neighbor states in the assignment share the same color



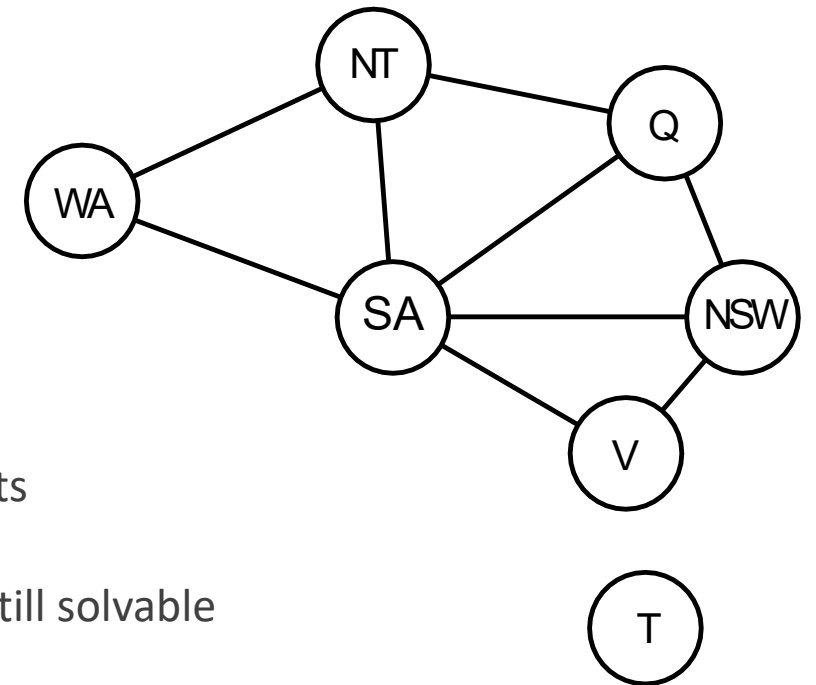
Constraint graph

Variables are nodes, constraints are arcs.

The graph structure can help the search (Tasmania)

You don't like this CSP? Here's another:

- Discrete, finite domains (boolean variables)
- Discrete, infinite domains (integers, like days)
- Discrete domains, linear constraints (solvable) vs. nonlinear constraints (undecidable, no general algorithm exist for solving this CSP)
- Continuous domains (real numbers, like time), linear constraints are still solvable (w. linear programming)



Constraints types

Unary

- One variable involved: $A \neq 1$

Binary

- Two variables involved (duh!): $A \leq B$

Global

- Any number of variables involved (not necessarily *all* variables)
- Alldiff(A,B,C,D)

Preference constraints

- Soft constraints that represent a preference (e.g., if possible, assign this over that)
- Modeled by a cost associated with each assignment (*constrained optimization problem*)
- E.g., linear programming

Absolute constraints

Constraint hyper-graph

For n-ary constraints it's not possible to build a graph where constraints are edges

- By definition, an edge connects two nodes only

e.g., cryptarithmic problem

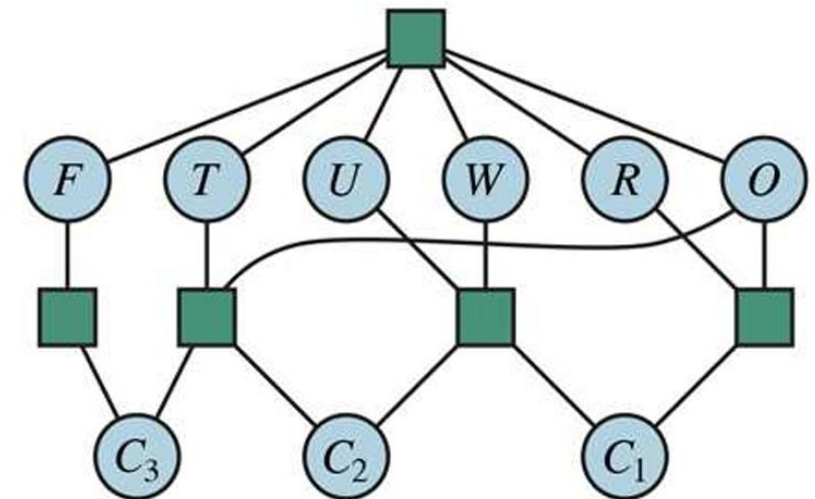
$$O + O = R + 10 C_1$$

$$C_1 + W + W = U + 10 C_2$$

$$C_2 + T + T = O + 10 C_3$$

$$C_3 = F$$

$$\begin{array}{r} T \quad W \quad O \\ + \quad T \quad W \quad O \\ \hline F \quad O \quad U \quad R \end{array}$$



Graph or hyper-graphs?

For constraints with a finite domain, you can always reduce a hyper-graph to a normal graph

- n-ary constraints are split into a set of binary constraints (auxiliary, hidden variables)

Useful if you want to solve CSP with algorithms for binary constraints only

Some popular constraints (e.g., Alldiff) benefit from ad-hoc algorithms

- No need to reduce them to binary constraints
- They are easier to read from a human perspective

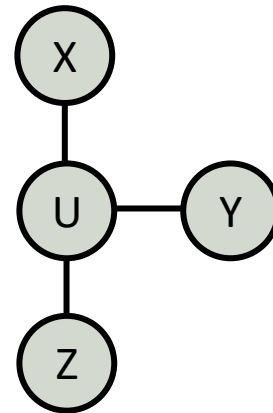
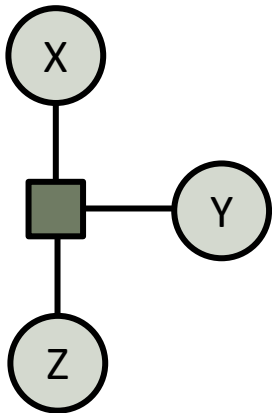
n-ary to binary

$$X + Y = Z, \quad X, Y, Z \in \{1, 2, 3\}$$

hidden variable $U \in \{(x, y, z) | x \in D_X, y \in D_Y, z \in D_Z, \quad x + y = z\}$

- Building the latent variable requires enumerating the cartesian product of the original domains

$U[1] = X, U[2] = Y, U[3] = Z \rightarrow 3$ binary constraints instead of a single 3-ary constraint



Some real-world examples

Chip design

Transportation

Scheduling

...

	lun 5/10				mar 6/10				mer 7/10				gio 8/10				ven 9/10			
Tutto il giorno																				
09	09:00 - 11:00		09:00 - 11:00		09:00 - 11:00	09:00 - 11:00	09:00 - 11:00	09:00 - 11:00	09:00 - 11:00	09:00 - 11:00		09:00 - 11:00		09:00 - 11:00		09:00 - 11:00	09:00 - 11:00	09:00 - 11:00	09:00 - 11:00	
10	ACCELERATED COMPUTING INFORMATICA (1) Fib I-Lab G. MENCAGLI		ALGORITHM ENGINEERING INFORMATICA (1) Fib C S. FERRAGINI		ACCELERATED COMPUTING INFORMATICA Fib M1	ALGORITHM ENGINEERING INFORMATICA Fib E	BUSINESS PROCESS MODELING DATA SCIENCE BUSINESS (2) Fib C	COMPUTER VISION INFORMATICA Fib C	ADVANCED LABORATORY NETWORK ANALYSIS DATA SCIENCE AND BUSINESS INFORMATICA (1) Fib H-Lab S. FERRAGINI	ALGORITHM ENGINEERING INFORMATICA (1) Fib C1		DATA MINING INFORMATICA (1) Fib C A. MONREALE		3D GEOMETRY MODELING & PROCESSING INFORMATICA Fib C1		ADVANCED SOFTWARE ENGINEERING INFORMATICA Fib C1	ICT RISK ASSESSMENT INFORMATICA Fib PS4	SCALABLE DISTRIBUTED COMPUTING INFORMATICA Fib C1		
11	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	11:00 - 13:00	
12	ADVANCED LABORATORY COMPLEX NETWORK ANALYSIS DATA SCIENCE (1) Fib M4	COMPETITIVE PROGRAMMING CONTESTS INFORMATICA Fib C A. MONREALE	DATA MINING INFORMATICA Fib C	ICT RISK ASSESSMENT INFORMATICA (1) Fib M-Lab	COMPUTATIONAL MATHEMATICS FOR TECHNOLOGIES AND DATA ANALYSIS INFORMATICA (1) Fib C	HUMAN LANGUAGE TECHNOLOGIES INFORMATICA (1) Fib C	MODELS FOR PROGRAMMING PARADIGMS INFORMATICA Fib C	ARTIFICIAL INTELLIGENCE FUNDAMENTALS INFORMATICA (1) Fib C	INFORMATION RETRIEVAL INFORMATICA (1) Fib L1	MODELS FOR PROGRAMMING PARADIGMS INFORMATICA F. VENTURINI, F. NARDINI	ADVANCED SOFTWARE ENGINEERING INFORMATICA Fib C	ARTIFICIAL INTELLIGENCE FUNDAMENTALS INFORMATICA Fib C1	FOUNDATION COMPUTING WEB INFORMATICA (1) Fib I-Lab F. BONCHI	SEMANTIC WEB INFORMATICA Fib I-Lab	GEOSPATIAL ANALYTICS DATA SCIENCE AND BUSINESS INFORMATICA Fib I-Lab M. NANNI, L. PAPPALARDO	SOFTWARE VERIFICATION: PRINCIPLES AND TECHNIQUES INFORMATICA (2) Fib C1				
13																				
14	14:00 - 16:00		14:00 - 16:00		14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	14:00 - 16:00	
15	3D GEOMETRY MODELING & INTELLIGENT PROCESSING FUNDAMENTALS INFORMATICA Fib M4		ARTIFICIAL INTELLIGENCE ANALYTICS DATA SCIENCE INFORMATICA Fib C		ADVANCED SOFTWARE ENGINEERING INFORMATICA (2) Fib C	DATA MINING INFORMATICA (1) Fib C1	SOFTWARE VERIFICATION: PRINCIPLES AND TECHNIQUES INFORMATICA A. MONREALE	ADVANCED SOFTWARE ENGINEERING INFORMATICA (1) Fib L1	COMPETITIVE PROGRAMMING AND CONTESTS INFORMATICA Fib C	COMPUTATIONAL MATHEMATICS FOR LEARNING TECHNOLOGIES AND DATA ANALYSIS INFORMATICA Fib C1	ADVANCED SOFTWARE ENGINEERING INFORMATICA (1) Fib C1	ARTIFICIAL INTELLIGENCE FUNDAMENTALS INFORMATICA Fib C1	FOUNDATION COMPUTING WEB INFORMATICA (1) Fib I-Lab	SEMANTIC WEB INFORMATICA Fib I-Lab	GEOSPATIAL ANALYTICS DATA SCIENCE AND BUSINESS INFORMATICA Fib I-Lab	SOFTWARE VERIFICATION: PRINCIPLES AND TECHNIQUES INFORMATICA (2) Fib C1	MODELS FOR PROGRAMMING PARADIGMS INFORMATICA (1) Fib C1			
16	16:00 - 18:00				16:00 - 18:00				16:00 - 18:00				16:00 - 18:00				16:00 - 18:00			
17	INFORMATION RETRIEVAL INFORMATICA (1) Fib C		INFORMATION RETRIEVAL INFORMATICA (1) Fib C		DISTRIBUTED ALGORITHMS INFORMATICA (1) Fib M1		DISTRIBUTED ALGORITHMS INFORMATICA (1) Fib M1		COMPUTER VISION INFORMATICA (2) Fib C1		DISTRIBUTED ALGORITHMS INFORMATICA (1) Fib L1		BUSINESS PROCESS MODELING DATA SCIENCE BUSINESS INFORMATICA Fib C		COMPUTER VISION INFORMATICA Fib C1		ICT RISK ASSESSMENT INFORMATICA Fib C1	SOFTWARE VERIFICATION: PRINCIPLES AND TECHNIQUES INFORMATICA Fib C1	ADVANCED SOFTWARE ENGINEERING INFORMATICA (1) Fib C1	FOUNDATION OF HUMAN LANGUAGE TECHNOLOGIES INFORMATICA (1) Fib C
	R. VENTURINI, F. NARDINI				G. PRENCIPE				A. CARTA, D. SHENAJ				A. CARTA, D. SHENAJ		F. BONCHI		A. CISTERMINO, A. COFFI			

How to search in a CSP

With atomic state the basic tool to use is **expansion**

- Selecting the node's successors

In a factored representation we can exploit the state structure

- Vectorial state representation

We have n variables

Each variable can take on d values (suppose)

States are defined by the values assigned so far

DFS in a CSP – a naive solution

Initial state

- the empty assignment, $\{ \}$

Actions:

- assign a value to an unassigned variable
- branching factor at the root = nd
- Branching factor at the second level = $(n - 1)d$

Every solution appears at depth n

- Issue: $n! d^n$ leaves vs. d^n possible complete assignments
- Fix: exploit the structure of the problem (commutativity, see next slide)

Backtracking search

DFS, but **all nodes at the same depth assign the same variable** (remove the $n!$ complexity)

- d^n leaves «only»!
- e.g., fix variable ordering beforehand

If at any point any constraint is broken → backtrack (restore most recent consistent assignment)

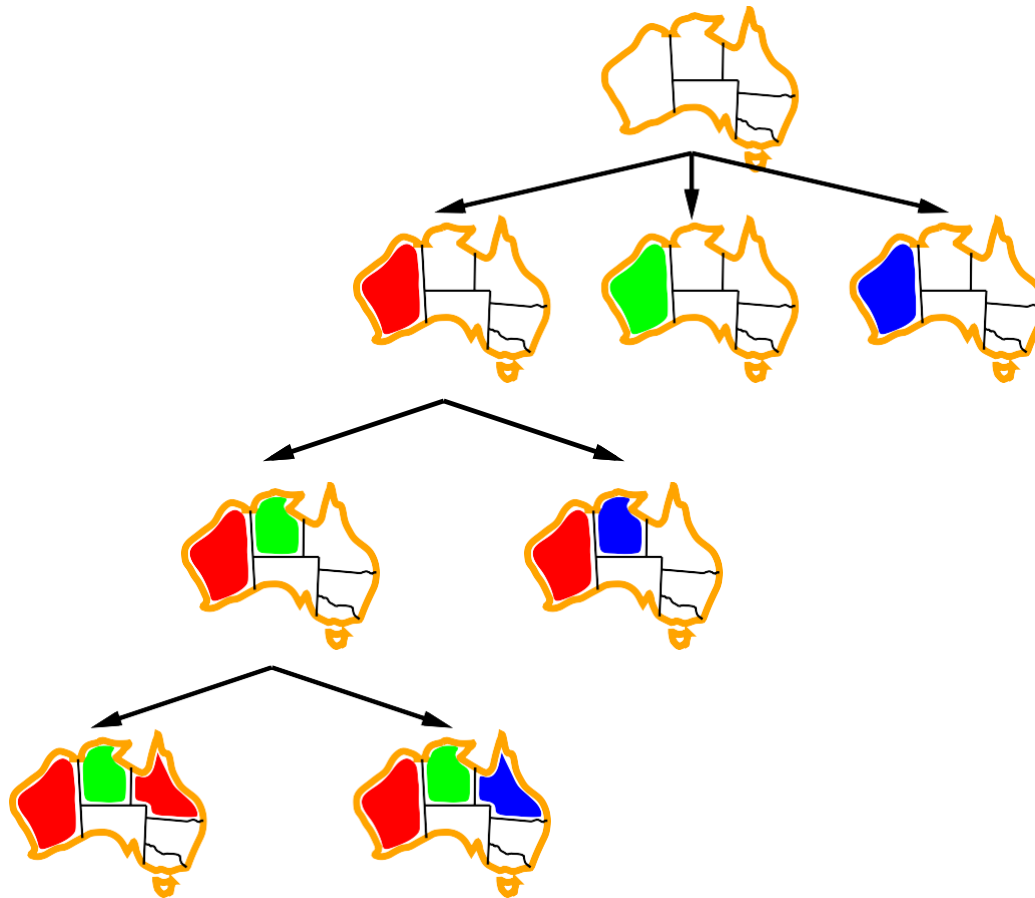
As soon as you reach a complete assignment, you found a solution

Backtracking search

```
function Backtracking-Search(csp) returns solution/failure
  return Recursive-Backtracking({ }, csp)

function Recursive-Backtracking(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← Select-Unassigned-Variable(Variables[csp], assignment, csp)
  for each value in Order-Domain-Values(var, assignment, csp) do
    if value is consistent with assignment given Constraints[csp] then
      add {var = value} to assignment
      result ← Recursive-Backtracking(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Backtracking example



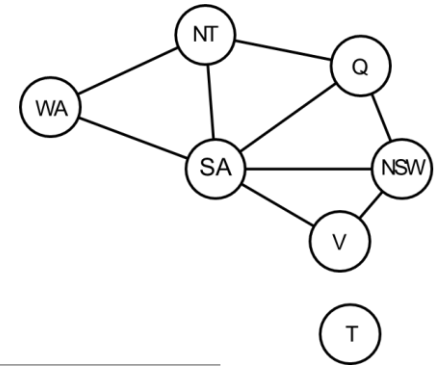
Uninformed algorithms in factored representations

With atomic states, uninformed algorithms did not leverage heuristics

In a factored representation, domain-independent heuristics do exist

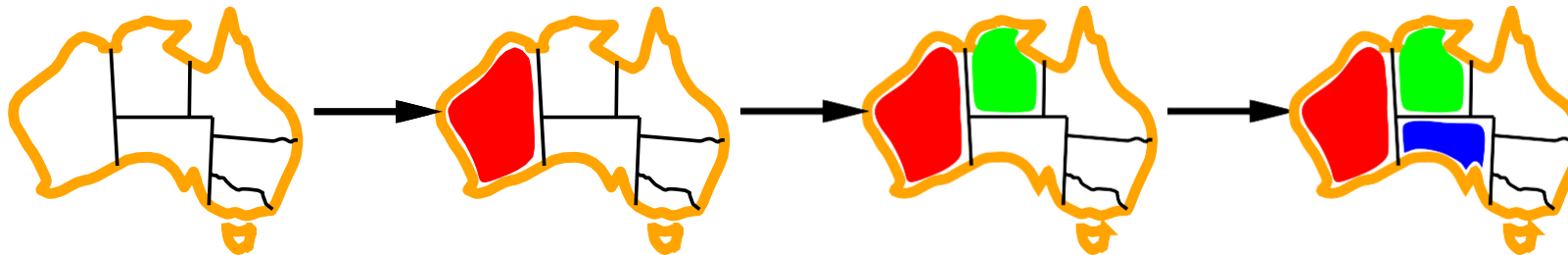
- They can speed up the search

Improvement Minimum Remaining Values (MRVs)

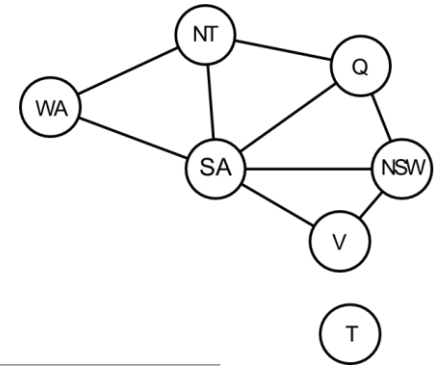


Choose variable with fewest legal values.

Failure-first, such that pruning occurs earlier



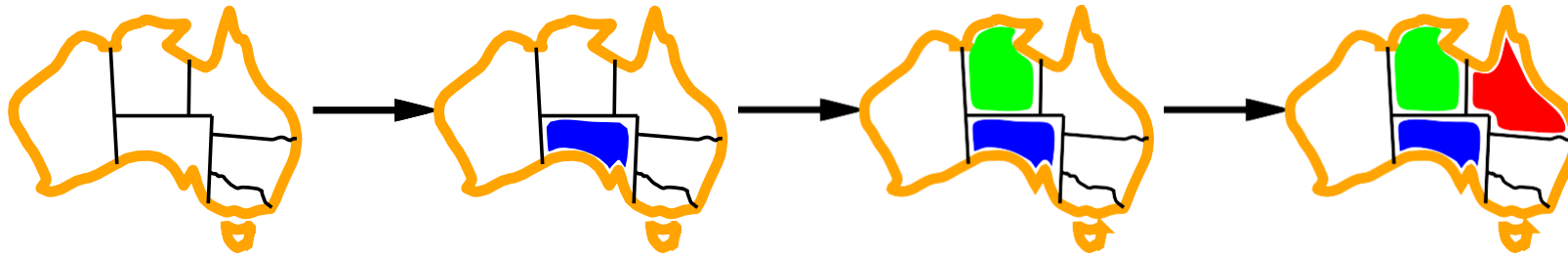
Improvement: Degree heuristic



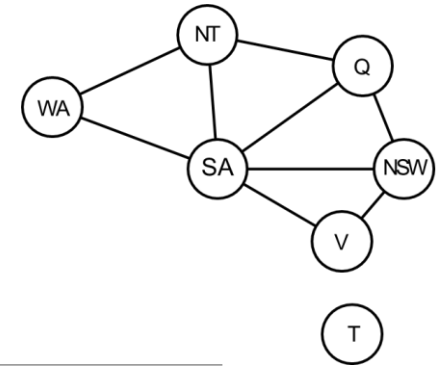
Choose variable involved in more constraints with unassigned variables

Reduce future branching size

Used when variables are tied according to MRVs



Improvement: Least-constraining values

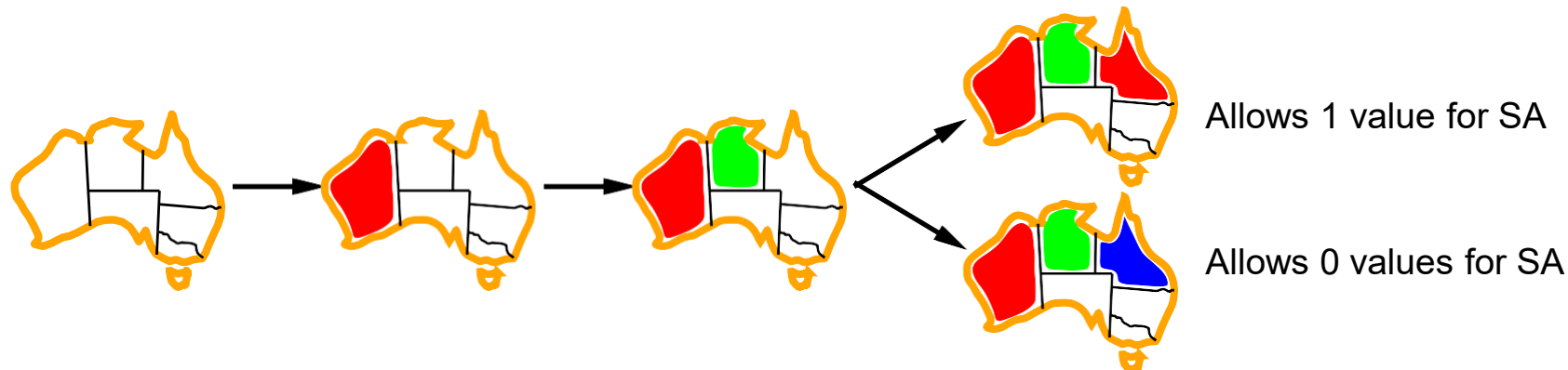


Choose a value assignment that rules out the fewest values in the other variables to be assigned

MRVs: **variable selection** triggers failure as early as possible (prune as early as possible)

LCVs: **variable assignment** keeps options open (do not enter a narrow, but possibly wrong path)

- They both try to quickly find a solution, from a different perspective



Inference by Constraint Propagation (CP)

Inference in CSP

- no search, just reduce the space of possible assignments
- a general technique to simplify the problem before search starts

CP reduces a variable's domain by looking at constraints

- Akin to some local search ideas

Can solve the problem in one go, if you are lucky

Node consistency

A simple example of CP

Remove any value in a variable's domain that breaks unary constraints

$$X = \{1,2,3,4\}, \quad X < 3 \rightarrow X = \{1,2\}$$

A *graph* is **node-consistent** if all variables are node-consistent

Always apply node consistency before search to get rid of unary constraints

How much does it cost?

- Linear complexity wrt the size of the variable's domain and the number of unary constraints

Arc Consistency

Another example of CP

A *variable* X is **arc-consistent** $\equiv$ for all constraints $X \rightarrow Y$ it holds: $\forall x \in X, \exists y \in Y \mid y \text{ allowed}$

A *graph* is **arc-consistent** if all variables are arc-consistent

How much does it cost?

- Quadratic complexity wrt the size of the variable's domain and linear complexity wrt the number of binary constraints

AC-3

How to make a graph arc-consistent? **AC-3** returns an arc-consistent graph, or fail

- Create a queue with all arcs
- Repeat until the queue is empty:
- Remove a random $X \rightarrow Y$ from the queue, make X arc-consistent wrt Y (reduce the domain of X by removing values that break constraints with Y)
- If any value in the domain of X is removed, add all $K \rightarrow X$ to the queue
- If at any point any variable is left with an empty domain: fail

$O(cd^3)$ for a CSP with c binary arcs

- cd^2 from arc consistency for all arcs
- Each arc can be added at most d times (if the domain of a node is reduced one value at a time)

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

queue $\leftarrow$ a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**

 (*X_i*, *X_j*) $\leftarrow$ POP(*queue*)

if REVISE(*csp*, *X_i*, *X_j*) **then**

if size of *D_i* = 0 **then return** false

for each *X_k* **in** *X_i*.NEIGHBORS - {*X_j*} **do**

 add (*X_k*, *X_i*) to *queue*

return true

function REVISE(*csp*, *X_i*, *X_j*) **returns** true iff we revise the domain of *X_i*

revised $\leftarrow$ false

for each *x* **in** *D_i* **do**

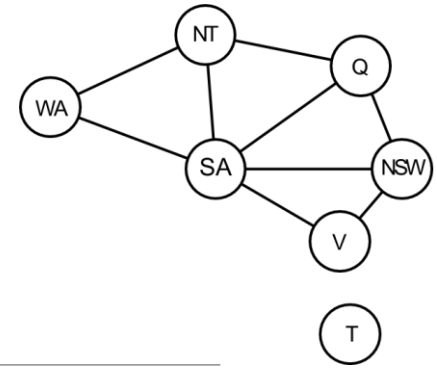
if no value *y* in *D_j* allows (*x*,*y*) to satisfy the constraint between *X_i* and *X_j* **then**

 delete *x* from *D_i*

revised $\leftarrow$ true

return *revised*

Improving search with inference: Forward checking



Whenever assigning a value to X
Make X arc-consistent wrt unassigned variables

If no values left in the domain: backtrack!

- We will not cover the many backtracking strategies that exist



WA	NT	Q	NSW	V	SA	T

Improvement via inference: Maintaining Arc Consistency

MAC augments forward checking by calling AC-3 after each assignment during search.
AC-3 is called with a queue containing all $Y \rightarrow X$ for all unassigned Y

If AC-3 fails: backtrack

MAC prunes more than forward checking as it recursively checks for inconsistencies

Local search for CSP

Complete-state formulation where each state assigns a value to every variable. Search changes the value of one variable at a time.

- So, initial state = random assignment of all variables

Min-conflicts heuristic: choose a variable that breaks some constraints → assign value that breaks the least constraints

- It brings us closer to the solution

```
function MIN-CONFLICTS(csp, max_steps) returns a solution or failure
  inputs: csp, a constraint satisfaction problem
           max_steps, the number of steps allowed before giving up

  current ← an initial complete assignment for csp
  for i = 1 to max_steps do
    if current is a solution for csp then return current
    var ← a randomly chosen conflicted variable from csp.VARIABLES
    value ← the value v for var that minimizes CONFLICTS(csp, var, v, current)
    set var = value in current
  return failure
```

Constraint weighting

Initially, all constraint share the same weight=1

At each step:

- Change the value of a variable s.t. total weight is minimized
- Increase (+1, or hyperparameter) the weight of each violated constraint

Weights are adapted during search, and more difficult constraints will receive larger weights

Guide the search to solve them, first

- large contribution to the minimization if you do not break them

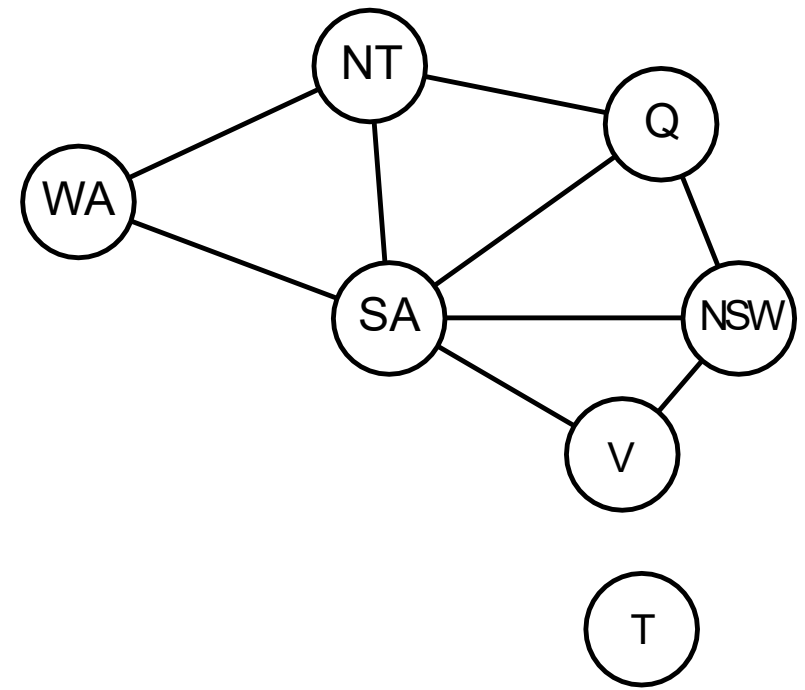
Exploit the problem structure

Even with a bag of million tricks (there are many more!), CSP remains very difficult in general

- d^n worst-case (as we have seen), the number of leaves

T and mainland are **independent subproblems**

- graph algorithms can identify sub-structures in the graph (**connected components**)
- Connected components are independent, solution is the union of components' solutions
- Scales as d^c , $c < n$ are the variables involved in the subproblem



There is a general result for a specific type of subproblem!

Tree-structured CSPs

If the constraint graph is a **tree** CSP costs $O(nd^2)$

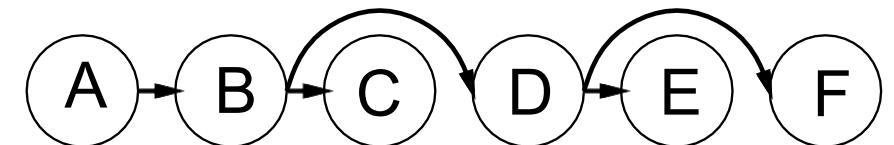
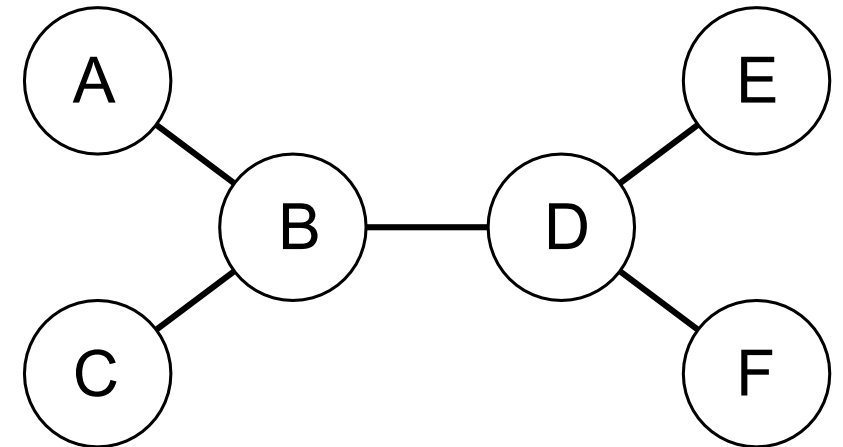
- **Linear** in n

Given a graph constraint without loops:

Search algorithm:

1. **Topological sort:** choose any variable as root, order variables such that each one is a child of its parent
2. Make the tree arc-consistent, return failure otherwise ($n-1$ arcs, d^2 values combinations to check per arc)
3. Assign to each node any value consistent with its parent, or fail

Never backtrack!



Planting trees where there are none

We can «quickly» solve tree-structured CSPs
Why not forcing a CSP to be tree-structured?

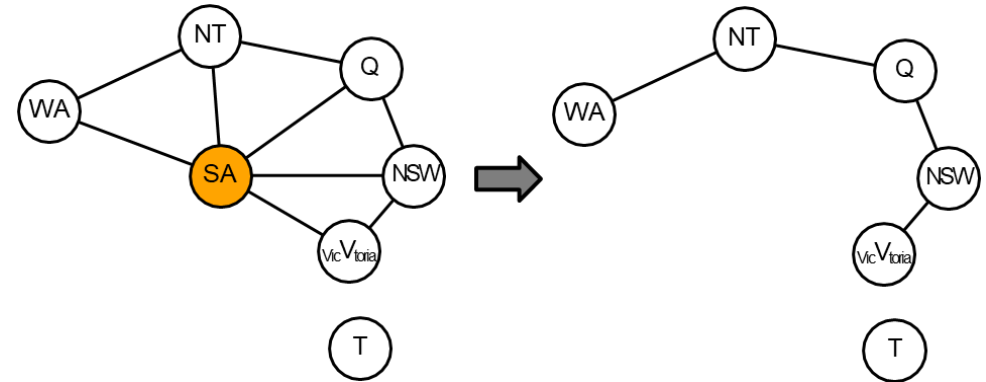
1. Assign a variable to a node
2. Make arcs consistent
3. Remove that variable

If the resulting graph can be made a tree, you can quickly check if it is solvable

Otherwise, you can try out another variable assignment or node selection

Suppose you try out c variables (**cutset conditioning**)

- $O(d^c (n - c)d^2)$





Upcoming – Games